

Fruit Catcher Repair Lab

Vibe Coding Chat History — Tasks 1 to 4

Name : Gagana B S

SRN: PES1UG25CS814

TASK 1 — Fix Floor-Miss Handling

I am completing Task 1 of the Fruit Catcher Repair Lab.

Please inspect the existing code and identify the bug in the floor-miss handling.

According to the README, the required behavior is:

1. When a normal fruit is caught by the basket, award +1 point.
2. When a fruit reaches the floor without being caught, deduct exactly 1 life.
3. A missed fruit must NOT increase the score.
4. When all 3 lives are lost, display the GAME OVER state and stop further game updates.
5. Pressing R after Game Over should restart the game with score = 0 and lives = 3.
6. Do not change unrelated game behavior.
7. Make the smallest necessary code changes.
8. Before editing, explain which file/function contains the bug and why.
9. After editing, summarize exactly what you changed.

Please inspect the existing implementation first rather than rewriting the game.

11:18 AM

-

Checkpoint Restored

- `Redo`

GitHub Copilot

-

Completed 17 steps in 1m 33s

`Auto routed task to GPT-6 Luna`

`Auto routed task to GPT-6 Luna`

I'll trace the existing floor-miss path and its game-over/restart handling first, then verify the smallest relevant

check before changing anything. I'll also read the Pyt

hon fact-grounded workflow required for this debugging task.

`Ran Get-Content 'C:\Users\Admin\.vscode\extensions\ms-...`

`Reviewed 3 files`

The owning code is `GameEngine.update()` in `game\_engine.py`: its catch branch correctly increments score, but

the floor-miss branch also increments score and never touch

es lives or game state. I'm checking the miss predicate and main loop now; the focused

disconfirming check is whether the game already transitions to Game Over elsewhere.

```
`Searched for test files and reviewed 3 files`  
`Fruit.is_missed()` confirms a miss is detected only after the fruit's center passes the screen height, and `main()` calls `update()` each frame. Nothing else changes lives or enters Game Over, so the bug is isolated to the miss branch in `GameEngine.update()`;  
I'm doing a tiny runtime repro of that branch before editing.  
`Ran pylanceRunCodeSnippet pylance mcp server (MCP Server)`
```

Fruit Catcher Repair Lab — Copilot Chat History Page 2

Before editing: the bug is specifically in `GameEngine.update()` in `game\_engine.py`. A floor miss currently adds a point and removes the fruit, while the file has no other life deduction or Game Over transition; the existing top-of-update guard and `reset()` already provide the freeze and restart behavior once state is set. I'll only replace that miss-branch scoring with a one-life deduction and set Game Over at zero, stopping the current loop there.

The direct repro couldn't import Pygame in the selected interpreter, so I'll validate the actual engine logic with a lightweight in-memory stub.

Edited

```
game_engine.py+4-1
```

```
`Ran pylanceRunCodeSnippet pylance mcp server (MCP Server)`
```

The focused engine check now passes all requested behaviors: catches score once, misses remove one life without changing score, the third miss enters a frozen Game Over state, and `R` resets score/lives/fruits. I'm checking the final diff and file diagnostics to confirm the edit stayed local.

```
`Ran git diff -- game/game_engine.py`
```

```
`Ran pylanceFileSyntaxErrors pylance mcp server (MCP Server)`
```

Checked `game\_engine.py`, no problems found

Fixed floor-miss handling in `game\_engine.py`: a missed fruit now deducts one life without changing the score.

When lives reach zero, the engine sets `GAME\_OVER` and stops processing that update. Catch scoring, the existing update freeze, and `R` restart behavior remain unchanged.

Validation passed for catch scoring, miss penalties, Game Over freeze, and restart; no syntax errors or file

diagnostics. Pygame isn't installed in the selected interpreter, so the behavior check used in-memory stubs. I'll inspect the current engine, item classes, and README as they stand now, since the engine may have changed since Task 1. Then I'll identify the narrowest place to distinguish hazards from normal fruit and describe the file/function edits before making them.

Read
Read
Read
Read skill `python-fact-grounded-coding`

TASK 2 — Add Hazards

The current implementation already stores every falling object in `self.fruits`, and its floor-miss branch costs a life for anything in that list. I'll add a `Hazard` subclass in `fruit.py` so it inherits the normal falling/position behavior but draws differently, then update

`GameEngine.update()` in `game\_engine.py` to occasionally spawn one, penalize only when it is caught, and avoid treating it as normal fruit or penalizing it for reaching the floor.

`reset()` already clears the shared list, and the existing game-state guard supports Game Over without further structural changes.

Generating patch (71 lines) in ,

Ran `pylanceRunCodeSnippet`

Completed with input: {

```
"workspaceRoot": "c:\\\\Users\\\\Admin\\\\64_fruit_catcher",
"workingDirectory": "c:\\\\Users\\\\Admin\\\\64_fruit_catcher",
"codeSnippet": "import sys\nimport types\nfrom collections import defaultdict\n\npygame =
types.ModuleType('pygame')\n\nclass Rect:\n    def __init__(self, *args, collides=
```

Fruit Catcher Repair Lab — Copilot Chat History Page 5

Ran `pylanceRunCodeSnippet`

Completed with input: {

```
"workspaceRoot": "c:\\\\Users\\\\Admin\\\\64_fruit_catcher",
"workingDirectory": "c:\\\\Users\\\\Admin\\\\64_fruit_catcher",
"codeSnippet": "import sys\nimport types\nfrom collections import defaultdict\n\nnclock = {'now': 1000}\n\npygame = types.ModuleType('pygame')\n\nclass Rect:\n    def __init__(self, x, y, width, height):\n        self.x, self.y, self.width, self.height = x, y, width, height\n    def colliderect(self, other):\n
```

```

return (self.x < other.x +
other.width and self.x + self.width > other.x\n and self.y < other.y + other.height and self.y +
self.height >
other.y)\npygame.Rect = Rect\npygame.time =
types.SimpleNamespace(get_ticks=lambda: clock['now'])\npygame.font =
types.SimpleNamespace(SysFont=lambda *args: None)\npygame.key =
types.SimpleNamespace(get_pressed=la
mbda: defaultdict(bool))\npygame.K_LEFT, pygame.K_a, pygame.K_RIGHT, pygame.K_d =
'left', 'a', 'right',
'd'\npygame.KEYDOWN, pygame.K_r = 'keydown', 'r'\n\nsys.modules['pyg
ame'] = pygame\n\nfrom game.fruit import Fruit, Hazard\n\nimport game.game_engine as
engine_module\n\nfrom
game.game_engine import GameEngine\n\nfrom types import SimpleNamespa
ce\n\nengine = GameEngine(600, 500)\n\nassert engine.spawn_delay == 750\n\nengine.score =
10\n\nclock['now'] =
1001\n\nengine.update()\n\nassert engine.spawn_delay == 680\n\nengine_m
odule.random.random = lambda: 0.0\n\nclock['now'] = 1680\n\nengine.update()\n\nassert
isinstance(engine.fruits[-1],
Hazard)\n\nassert 4.3 <= engine.fruits[-1].speed <= 6.8\n\n\neng
ine.score = 100\n\nclock['now'] = 1681\n\nengine.update()\n\nassert engine.spawn_delay ==
400\n\nengine_module.random.random = lambda: 1.0\n\nclock['now'] =
2080\n\nengine.update()\n\n
assert type(engine.fruits[-1]) is Fruit\n\nassert 5.5 <= engine.fruits[-1].speed <=
8.0\n\n\nengine.reset()\n\nassert
(engine.score, engine.lives, engine.spawn_delay, len(engine
e.fruits)) == (0, 3, 750, 0)\n\nmissed = Fruit(600)\n\nmissed.x, missed.y = 0,
501\n\nengine.fruits.append(missed)\n\nengine.update()\n\nassert (engine.score, engine.lives) ==
(0,
2)\n\n\nengine.reset()\n\nhazard = Hazard(600)\n\nhazard.x = engine.basket.x + 40\n\nhazard.y =
engine.basket.y + 10 -
hazard.speed\n\nengine.fruits.append(hazard)\n\nengine.update()
\n\nassert (engine.score, engine.lives) == (0, 2)\n\n\nprint('PASS: score-scaled spawn/speed caps
for normal and hazard
spawns; Task 1 miss and Task 2 hazard penalties/reset pr
eserved')",
"timeout": 30000
}

```

The focused test passes at low and high scores: the delay decreases from 750 ms to 680 ms at 10 points and

clamps at 400 ms; both spawned hazard and normal fruit receive

the score-based speed range, capped at +1.5 px/frame. It also confirms reset restores the

starting delay and the

Task 1 miss and Task 2 hazard penalties still behave as before. I'm doing a final syntax/diff check.

Ran terminal command: `git diff -- game/game_engine.py game/fruit.py`

Ran ``pylanceFileSyntaxErrors``

Completed with input: {

"workspaceRoot": "c:\\\\Users\\Admin\\64_fruit_catcher",

"fileUri": "file:///c:/Users/Admin/64_fruit_catcher/game/game_engine.py"

}

Ran ``pylanceFileSyntaxErrors``

Completed with input: {

"workspaceRoot": "c:\\\\Users\\Admin\\64_fruit_catcher",

"fileUri": "file:///c:/Users/Admin/64_fruit_catcher/game/fruit.py"

}

Checked , , 1 problem found

Updated ``game_engine.py`` to reduce spawn delay by 7 ms per point, down to a 400 ms

floor, and pass a speed

bonus to each newly spawned item. Updated ``fruit.py`` so both ``F`

Fruit Catcher Repair Lab — Copilot Chat History Page 6

``ruit`` and its ``Hazard`` subclass apply that bonus to their existing random speed range, capped by the engine at

+1.5 px/frame. Restart restores the 750 ms starting delay.

Focused tests passed for normal fruit and hazard speeds, both difficulty caps, reset, Task 1 misses, and Task 2

hazard penalties. Both files have no syntax errors. Pylanc

e still reports ``pygame`` unresolved in the selected interpreter; the behavior test used a lightweight stub. No Task

4 changes were made.

Made changes.

TASK 4 — Fruit Splash Particles

I'll inspect the current engine, fruit classes, and main loop first, especially how ``update()`` behaves after Game

Over and how Task 3 passes speed bonuses to both item ty

pes. That will determine where particles should age so the Game Over freeze remains intact while effects still

expire.

Read

Read

Read

Read , lines 38 to 86

Read skill `python-fact-grounded-coding`

The current `main()` keeps calling `engine.update()` even during Game Over, but

`GameEngine.update()` returns

immediately in that state. I'll add a small `Particle` class

in `game/particle.py`, then change only `GameEngine` initialization, update, render, and

reset: emit colored

particles on normal catches and floor impacts, age/remove th

em before the Game Over guard, draw them, and clear them on restart. This preserves the
existing score/life and

hazard branches while ensuring effects expire even after G
ame Over.

Generating patch (73 lines) in ,

Ran `pylanceRunCodeSnippet`

Completed with input: {

"workspaceRoot": "c:\\\\Users\\\\Admin\\\\64_fruit_catcher",

"workingDirectory": "c:\\\\Users\\\\Admin\\\\64_fruit_catcher",

"codeSnippet": "import sys\nimport types\nfrom collections import defaultdict\n\nclock =\n{'now': 1000}\npygame

= types.ModuleType('pygame')\n\nclass Rect:\n def __init__

(self, x, y, width, height):\n self.x, self.y, self.width, self.height = x, y, width, height\n def

colliderect(self, other):\n

return (self.x < other.x +

other.width and self.x + self.width > other.x\nand self.y < other.y + other.height and self.y +

self.height >

other.y)\npygame.Rect = Rect\npygame.time =

types.SimpleNamespace(get_ticks=lambda: clock['now'])\npygame.font =

types.SimpleNamespace(SysFont=lambda *args: None)\npygame.key =

types.SimpleNamespace(get_pressed=la

mbda: defaultdict(bool))\npygame.K_LEFT, pygame.K_a, pygame.K_RIGHT, pygame.K_d =

'left', 'a', 'right',

'd'\npygame.KEYDOWN, pygame.K_r = 'keydown', 'r'\npygame.draw_calls = []\n

pygame.draw = types.SimpleNamespace(circle=lambda *args:

draw_calls.append(args))\n\nsys.modules['pygame']

= pygame\n\nfrom types import SimpleNamespace\nfrom game.fruit i

mport Fruit, Hazard\nfrom game.game_engine import GameEngine\nfrom game.particle

import Particle\n\nengine

= GameEngine(600, 500)\n\nfruit = Fruit(600)\n\nfruit.x = engine.ba

sket.x + 45\n\nfruit.y = engine.basket.y + 10 -

fruit.speed\n\nengine.fruits.append(fruit)\n\nengine.update()\n\nassert

(engine.score, engine.lives, len(engine.particles)) == (1,

```
3, 8)\nassert all((particle.color == fruit.color and particle.life == 24 for particle in
engine.particles)\nengine.particles[0].render(None)\nassert draw_calls[-1][1] ==
```

Fruit Catcher Repair Lab — Copilot Chat History Page 7

```
fruit.color\n\nfor _ in range(24):\n engine.update()\nassert not engine.particles\n\nmissed
= Fruit(600)\nmissed.x,
missed.y = 0, 501\nengine.fruits.append(missed)\nen
gine.update()\nassert (engine.score, engine.lives, len(engine.particles)) == (1, 2, 8)\nassert
all((particle.y ==
engine.height - 25 for particle in engine.particles)\n\nh
azard = Hazard(600)\nhazard.x = engine.basket.x + 45\nhazard.y = engine.basket.y + 10 -
hazard.speed\nengine.fruits.append(hazard)\nengine.update()\nassert (engine.score,
engine.lives) == (1, 1)\nengine.lives = 1\nlast_miss = Fruit(600)\nlast_miss.x, last_miss.y =
0,
501\nengine.fruits.append(last_miss)\nengine.update()\nassert engine.g
ame_state == 'GAME_OVER'\nfor _ in range(24):\n engine.update()\nassert not
engine.particles\nengine._create_splash(20, 20, (255, 0,
0))\nengine.handle_event(SimpleNam
espace(type='keydown', key='r'))\nassert (engine.score, engine.lives, engine.game_state,
len(engine.fruits),
len(engine.particles)) == (0, 3, 'PLAYING', 0, 0)\nprint('PAS
S: catch/floor splashes, color and rendering, particle expiry during Game Over,
hazard/scoring/lives, and restart
clearing')",
"timeout": 30000
}
```

The focused test passes: normal catches and floor impacts each create eight fruit-colored particles, particles

render and expire after 24 updates, and they continue aging

after Game Over. It also confirms score/life behavior, hazard catches, and restart clearing remain intact. I'm

checking syntax and the final diff now.

Ran terminal command: git diff -- game/game_engine.py game/particle.py

Ran `pylanceFileSyntaxErrors`

Completed with input: {

"workspaceRoot": "c:\\\\Users\\Admin\\64_fruit_catcher",

"fileUri": "file:///c:/Users/Admin/64_fruit_catcher/game/game_engine.py"

}

Ran `pylanceFileSyntaxErrors`

Completed with input: {

"workspaceRoot": "c:\\\\Users\\Admin\\64_fruit_catcher",

```
"fileUri": "file:///c:/Users/Admin/64_fruit_catcher/game/particle.py"
```

```
}
```

Checked , , 1 problem found

Added `particle.py` with lightweight particles that move, shrink, and expire after 24 updates.

In

`game\_engine.py`, normal catches and floor impacts now emit eight partic

les using the fruit's color; particles are rendered, continue aging during Game Over, and are cleared on restart.

The focused test passed for splash creation, color, expiry, existing score/life and hazard behavior, and restart.

Syntax checks passed. Pylance still reports that `pygame

` is unresolved in the selected interpreter.